

Project 5: High-Performance HTTP Server

Detailed stage-wise build plan for a placement-grade C++ networking and performance project

Target outcome: Build a high-performance HTTP/1.1 server from low-level TCP sockets to demonstrate networking, C++, Linux, concurrency, protocol handling, benchmarking and performance optimization.

1. Project Scope

Project: High-Performance HTTP Server

Recommended stack: C++17/20, Linux, POSIX sockets, CMake, threads, optional epoll, GoogleTest/Catch2, wrk or another load generator, Docker.

Core user story: A client connects over TCP, sends a valid HTTP request, the server parses it, routes it to a handler and returns a standards-compliant response. The server must handle many concurrent connections efficiently and provide reproducible performance measurements.

What makes it resume-worthy

- Do not use a web framework. The learning and resume value comes from implementing the networking and server architecture yourself.
- Build a minimal correct server first, then introduce concurrency and performance features one at a time.
- Benchmark every major change so the final project can demonstrate measured improvement rather than merely listing features.

2. Target Architecture

Client -> TCP socket -> connection handling -> HTTP parser -> router -> handler -> response serializer -> socket.

The performance phase can replace a blocking event loop with a worker-pool/event-driven design and optionally Linux epoll.

Component	Purpose	Suggested implementation
Listener	Accept incoming TCP connections	socket/bind/listen/accept
Connection layer	Read/write bytes reliably	recv/send with partial-I/O handling
HTTP parser	Parse request line/headers/body	Custom parser + validation
Router	Map method/path to handler	Method + path dispatch table
Handlers	Produce application responses	Static files + simple REST endpoints
Concurrency	Serve many clients	Thread pool; optional epoll
Benchmarking	Measure performance	wrk + custom scripts

3. Stage-Wise Build Plan

Stage 0 - Design and Environment (0.5 day)

- Set up CMake, compiler warnings, sanitizers and automated tests.

- Define the supported HTTP subset before coding: methods, HTTP version, headers, body size and status codes.
- Create modules for socket handling, parsing, routing, response generation and benchmarking.

Stage 1 - Raw TCP Server (1-2 days)

- Create a listening socket, bind to a port, listen and accept connections.
- Read bytes from a client and return a simple response.
- Handle partial reads/writes and clean connection closure.
- Use a command-line client such as curl for repeatable smoke tests.

Exit criteria: curl can connect repeatedly and receive a valid HTTP response without crashes or malformed output.

Stage 2 - HTTP/1.1 Parser and Response Builder (2-3 days)

- Parse request method, request target, HTTP version and headers.
- Implement validation and clear 400/404/405/413/500-style error responses where appropriate.
- Implement Content-Length and/or chunked behavior only if you can test it correctly.
- Build a response serializer with status line, headers and body.

Stage 3 - Routing and Application Layer (1-2 days)

- Implement method/path routing such as GET /, GET /health and GET /static/.
- Add a small JSON REST endpoint to demonstrate application-level routing.
- Add static-file serving with path traversal protection.
- Write parser and routing unit tests before optimizing.

Stage 4 - Baseline Concurrency (2-3 days)

- Implement a simple thread-per-connection or single-worker baseline only as a comparison point.
- Measure requests/sec, average latency, p50/p95/p99 latency and CPU utilization.
- Document where the baseline stops scaling as concurrent clients increase.

Stage 5 - Thread-Pool Architecture (2-3 days)

- Introduce a bounded worker pool and task queue.
- Separate socket acceptance from request processing.
- Control queue growth and handle overload gracefully.
- Benchmark different worker counts to find the best throughput/latency trade-off.

Benchmark checkpoint: Keep the original baseline. The strongest resume result will compare the optimized server against a measured baseline rather than quoting an isolated throughput number.

Stage 6 - Persistent Connections and Efficient I/O (2-3 days)

- Support HTTP keep-alive where appropriate.
- Avoid unnecessary connection setup/teardown.

- Reduce redundant allocations and copies in request processing.
- Measure the effect of keep-alive and buffering separately.

Stage 7 - Event-Driven I/O with epoll (Optional Advanced Stage, 2-4 days)

- Use Linux epoll to manage many sockets without blocking one thread per connection.
- Define a non-blocking state machine for reads and writes.
- Compare thread-pool-only and event-driven architectures under high concurrency.
- Only include this stage if the earlier stages are stable; correctness comes before benchmark numbers.

Stage 8 - Profiling and Performance Optimization (2-3 days)

- Profile CPU time and allocation hotspots.
- Investigate parser overhead, locking, system calls, copies and file I/O.
- Apply one optimization at a time and rerun the same workload.
- Track throughput, p99 latency and CPU utilization after each change.

Stage 9 - Testing, Hardening and Demo (2 days)

- Add malformed-request, concurrency, large-payload and disconnect tests.
- Run AddressSanitizer/UndefinedBehaviorSanitizer and fix reported issues.
- Containerize the server and provide one-command build/run instructions.
- Create a benchmark script that reproduces the final headline metrics.

4. Metrics to Collect for the Resume

Metric	How to measure	Useful resume consequence
Throughput	Requests/sec at fixed payload/workload	X K req/s
p99 latency	Tail latency under concurrency	Y ms p99
Concurrency	Increase active clients until saturation	Supported Z concurrent connections
CPU	Process/system monitoring	Y% lower CPU
Baseline delta	Compare before/after optimization	X% throughput improvement
Payload scaling	Run several response/request sizes	Stable performance up to X KB/MB

5. Suggested Repository Structure

```

http-server/
CMakeLists.txt
include/ net/ http/ router/ server/
src/ net/ http/ router/ server/
tests/ unit/ integration/ load/
benchmarks/
docker/
docs/architecture.md
README.md

```

6. Interview Topics You Must Be Able to Explain

- TCP vs HTTP: what each layer provides and how a browser request reaches your server.
- Blocking vs non-blocking sockets and why epoll helps at high connection counts.
- Thread-per-connection vs bounded thread pools and their CPU/memory trade-offs.
- HTTP request framing, Content-Length, keep-alive and partial reads/writes.
- Why p99 latency matters and how benchmark methodology can produce misleading results.
- Race conditions, synchronization, queue backpressure and graceful shutdown.
- How you found the bottleneck using profiling rather than guessing.
- Security basics relevant to a server: path traversal, request-size limits and malformed input.

7. Final Resume Target

Ideal final project title: High-Performance HTTP Server | C++, Linux, TCP/IP, HTTP/1.1, Multithreading, epoll, Performance Engineering

Target bullets: 2-3 bullets using the ABC format. Example structure: *Engineered an HTTP/1.1 server in C++ from raw TCP sockets with request parsing, routing and thread-pool concurrency, achieving XK requests/sec at Y ms p99 latency with Z concurrent connections.*

Do not fabricate metrics. X/Y/Z are placeholders until the benchmark suite produces real measurements. Preserve benchmark commands, machine configuration and raw results so the final resume claims are reproducible and defensible.